

UNIVERSITÉ PROTESTANTE AU CONGO
FACULTÉ DES SCIENCES INFORMATIQUES
KINSHASA/LINGWALA

TRAVAIL PRATIQUE

FasiChatClassRoom

Matière : PROGRAMMATION WEB PHP

Conçu par les étudiants :

1. KAGHOMA KITSIPA John
2. KIPAKA BAKARI Michel
3. MATHE MWERO Ben

Promotion : L2 LMD

Année Académique : 2025-2026

Table des matières

1	Introduction	2
2	Objectifs	2
3	Technologies utilisées	2
4	Architecture du projet	3
4.1	Structure MVC	3
4.2	Front Controller	3
4.3	Hiérarchie des classes utilisateur	4
5	Base de données	4
6	Fonctionnalités détaillées	5
6.1	Messagerie en temps réel	5
6.2	Mur pédagogique	6
6.3	Valve (tableau d'affichage)	6
6.4	Convocations	6
6.5	Upload de fichiers	6
7	Sécurité	7
8	Problèmes rencontrés	7
8.1	Chemin d'upload incorrect	7
8.2	Erreur de constructeur Doyen/Vice-Doyen	8
8.3	Message avec fichier mais sans texte	8
8.4	Mauvais type de messages	8
8.5	Absence de vérification d'auth	8
9	Conclusion	8
9.1	Ce qui fonctionne	8
9.2	Pistes d'amélioration	9

Introduction

FasiChat est une application web qui permet aux étudiants, enseignants et administrateurs d'une faculté de communiquer facilement. Le projet est né d'un besoin simple : remplacer les groupes WhatsApp et les annonces papier par une plateforme unique, organisée et sécurisée.

L'application permet :

- d'échanger des messages privés ou publics
- de publier des annonces officielles (Valve)
- d'afficher un mur pédagogique par cours
- d'envoyer des convocations aux enseignants
- de partager des fichiers (PDF, images, audio, vidéo)

Ce rapport détaille l'architecture, les fonctionnalités et les choix techniques du projet.

Objectifs

L'objectif principal était de créer un espace de communication unique pour toute la faculté, avec des rôles bien définis :

- **Étudiant** : peut discuter en groupe (promotion), voir le mur des cours, envoyer des messages privés aux camarades, et consulter les annonces du Valve.
- **Enseignant / Assistant** : peut publier sur le mur pédagogique, discuter en public avec les étudiants, recevoir des convocations, et envoyer des messages privés aux collègues.
- **Doyen / Vice-Doyen** : peut convoquer des réunions, échanger des messages confidentiels, et consulter tous les espaces.
- **Apparitaire** : gère le tableau d'affichage (Valve) : publie, modifie et supprime les annonces officielles.

Technologies utilisées

Le projet utilise uniquement des technologies natives, sans framework :

- **PHP 8** côté serveur (POO, PDO, sessions)
- **MySQL** pour la base de données

- **JavaScript** (Fetch API, DOM) pour le côté client
- **CSS3** (Flexbox, variables CSS, dégradés)
- **Apache** (mod_rewrite pour le routage)
- **HTML5** pour les templates

Le choix de ne pas utiliser de framework était volontaire : cela permet de mieux comprendre chaque brique du projet et de montrer notre maîtrise des concepts fondamentaux.

Architecture du projet

Structure MVC

Le projet suit une architecture MVC (Modèle-Vue-Contrôleur) maison : les **Modèles** gèrent les données (Utilisateur, Message, Fichier...), les **Vues** affichent le HTML (login, dashboards, valve), et les **Contrôleurs** traitent les requêtes.

Front Controller

Toutes les requêtes passent par `public/index.php`. Le fichier `.htaccess` redirige les URLs vers ce fichier.

```
1 $path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
2 $path = str_replace('/FasiChatClassroom/public', '', $path);
3 $path = trim($path, '/');
4
5 switch ($path) {
6     case 'login':
7         require __DIR__ . '/../views/login.php';
8         break;
9     case 'message-send':
10         $msgModel = new Message($db);
11         $success = $msgModel->envoyer(...);
12         sendJsonResponse(['success' => $success]);
13         break;
14     default:
15         require __DIR__ . '/../views/login.php';
16 }
```

Listing 1 – Le routeur (`public/index.php`)

Hiérarchie des classes utilisateur

Les utilisateurs sont représentés par une hiérarchie de classes PHP : la classe de base `Utilisateur` est **abstraite** (on ne peut pas créer un « utilisateur » générique). `CadreAdministratif` ajoute la méthode `convoquer()` partagée par le Doyen et le Vice-Doyen.

```

1 abstract class Utilisateur {
2     protected $id, $nom, $prenom, $email, $role;
3     public function __construct($data = []) { ... }
4     public static function authentifier($db, $email, $password) {
5         ... }
6 }
7 // Heritiers : Etudiant, Enseignant (et Assistant),
8 //             CadreAdministratif (abstrait),
9 //             - Doyen, ViceDoyen,
10 //             Apparitaire

```

Listing 2 – Classes utilisateur

Base de données

La base `fasichat` contient 7 tables reliées par des clés étrangères. Voici les plus importantes :

```

1 -- Utilisateurs (7 roles)
2 CREATE TABLE utilisateurs (
3     id INT AUTO_INCREMENT PRIMARY KEY,
4     nom VARCHAR(100), prenom VARCHAR(100),
5     email VARCHAR(100) UNIQUE,
6     mot_de_passe VARCHAR(255), role VARCHAR(50)
7 );
8
9 -- Messages (type : 'prive', 'public', 'mur', 'convocation')
10 CREATE TABLE messages (
11     id INT AUTO_INCREMENT PRIMARY KEY,
12     expéditeur_id INT,
13     contenu TEXT,
14     date_envoi DATETIME DEFAULT CURRENT_TIMESTAMP,
15     type VARCHAR(20),
16     cours_id INT NULL, promotion_id INT NULL,
17     fichier_id INT NULL
18 );

```

```

19
20 -- Annonces (Value)
21 CREATE TABLE annonces (
22     id INT AUTO_INCREMENT PRIMARY KEY,
23     titre VARCHAR(255), contenu TEXT,
24     date_publication DATETIME DEFAULT CURRENT_TIMESTAMP,
25     auteur_id INT
26 );

```

Listing 3 – Tables principales

Les messages privés sont liés à un destinataire via la table `messages_destinataires`. Les messages publics sont liés à une promotion, les messages de mur sont liés à un cours.

Fonctionnalités détaillées

Messagerie en temps réel

Les messages sont envoyés et reçus sans rechargement de page. Toutes les 3 secondes, le navigateur demande les nouveaux messages au serveur (technique de **polling AJAX**).

```

1 class Message {
2     private $db;
3     public function envoyer($expediteurId, $contenu, $type,
4         $destinataireId = null, $coursId = null,
5         $promotionId = null, $fichierId = null) {
6         $this->db->beginTransaction();
7         // Insertion du message
8         $query = "INSERT INTO messages (...) VALUES (...)";
9         $stmt->execute([...]);
10        // Si prive, on lie le destinataire
11        if ($type === 'prive') {
12            $queryDest = "INSERT INTO messages_destinataires
13                (message_id, destinataire_id) VALUES (...)
14                ";
15            $stmtDest->execute([...]);
16        }
17        $this->db->commit();
18        return true;
19    }

```

Listing 4 – Envoi d'un message (*co^te' serveur*)

Mur pédagogique

Chaque cours a un mur où l'enseignant publie des annonces, rappels ou questions. Les étudiants voient ces publications dans leur sidebar. Les messages de type **mur** sont filtrés par `cours_id`.

Valve (tableau d'affichage)

Le Valve affiche toutes les annonces officielles, triées de la plus récente à la plus ancienne. Seul l'Apparitaire peut publier, mais tout le monde peut les consulter.

```

1 $stmtAnnonces = $db->query("
2     SELECT a.*, u.nom, u.prenom, u.role
3     FROM annonces a
4     JOIN utilisateurs u ON a.auteur_id = u.id
5     ORDER BY a.date_publication DESC
6 ");
7 $annonces = $stmtAnnonces->fetchAll();

```

Listing 5 – Affichage des annonces

Convocations

Le Doyen et le Vice-Doyen peuvent convoquer les enseignants. Le système crée un message de type **convocation** et une entrée dans la table **convocations** (objet, date, lieu).

Upload de fichiers

Le téléversement se fait en deux étapes : (1) l'utilisateur sélectionne un fichier, le JavaScript l'envoie au serveur qui le stocke avec un nom unique ; (2) le `fichier_id` est joint au message lors de l'envoi. Les images sont compressées à 800px de largeur pour économiser de l'espace.

```

1 public function upload($fileArray) {
2     $extension = strtolower(pathinfo($name, PATHINFO_EXTENSION));
3     $safeName = uniqid('file_', true) . '.' . $extension;
4     $destPath = __DIR__ . '/../public/assets/uploads/' .
5         $safeName;

```

```
6     if (in_array($extension, ['jpg', 'jpeg', 'png'])) {
7         $this->compressImage($fileArray['tmp_name'], $destPath,
8             $ext);
9     } else {
10         move_uploaded_file($fileArray['tmp_name'], $destPath);
11     }
12     // Enregistrement en base...
13     return $this->db->lastInsertId();
14 }
```

Listing 6 – Upload se'curise' de fichier

Sécurité

- **Mots de passe** : hachés avec `password_hash()` (bcrypt). Vérification avec `password_verify()`.
- **Injections SQL** : toutes les requêtes utilisent des requêtes préparées PDO avec paramètres nommés.
- **XSS** : les données affichées sont échappées avec `htmlspecialchars()` côté PHP et `escapeHTML()` côté JavaScript.
- **Fichiers** : noms randomisés avec `uniqid()` pour éviter le path traversal.
- **Session** : chaque page vérifie la session et le rôle de l'utilisateur avant d'afficher le contenu.

Problèmes rencontrés

Chemin d'upload incorrect

Problème : les fichiers étaient stockés dans `src/public/assets/uploads/` au lieu de `public/assets/up`. La constante `__DIR__` pointant vers `src/Models/`, un seul `../` remontait seulement à `src/`.

Solution : utiliser `../../` au lieu de `../`.

```
1 // Avant (faux)
2 $destPath = __DIR__ . '/../' . $uploadDir . $safeName;
3 // Apres (corrige)
4 $destPath = __DIR__ . '/../../' . $uploadDir . $safeName;
```


Erreur de constructeur Doyen/Vice-Doyen

Problème : `new Doyen($db, $_SESSION['user'])` passait l'objet PDO comme données utilisateur. Le constructeur `Doyen($data)` interprétait `$db` comme `$data`, ce qui provoquait une erreur.

Solution : `new Doyen($_SESSION['user'])` (un seul paramètre).

Message avec fichier mais sans texte

Problème : la validation exigeait un texte non vide. Si l'utilisateur joignait un fichier sans écrire de message, l'envoi échouait.

Solution : autoriser l'envoi quand un fichier est présent.

```
1 // Avant
2 if (empty(trim($contenu))) { throw new Exception(...); }
3 // Apres
4 if (empty(trim($contenu)) && empty($fichierId)) { throw new
    Exception(...); }
```

Mauvais type de messages

Problème : les messages étudiant-enseignant étaient en `type='prive'` au lieu de `type='public'`. Ils n'étaient visibles que par deux personnes au lieu de toute la promotion.

Solution : modifier les appels JavaScript pour utiliser `'public'` avec l'ID de la promotion.

Absence de vérification d'auth

Problème : les pages `dashboard_apparitaire.php` et `dashboard_admin.php` n'avaient aucune vérification de session.

Solution : ajouter `session_start()` et une redirection si l'utilisateur n'est pas connecté.

Conclusion

Ce qui fonctionne

Le projet répond aux objectifs fixés :

- messagerie temps réel entre tous les acteurs
- distinction claire entre messages privés, publics et de mur
- Valve (annonces) géré par l'Apparitaire
- convocations pour les enseignants
- upload et compression de fichiers
- interface adaptée à chaque rôle
- protection contre les injections SQL et XSS

Pistes d'amélioration

Quelques pistes pour aller plus loin :

- remplacer le polling par des WebSockets (latence réduite)
- ajouter des notifications (email ou push)
- interface mobile responsive dédiée
- système de « lu » pour les messages
- administration des utilisateurs (CRUD)

Fin du rapport